
Design Concepts

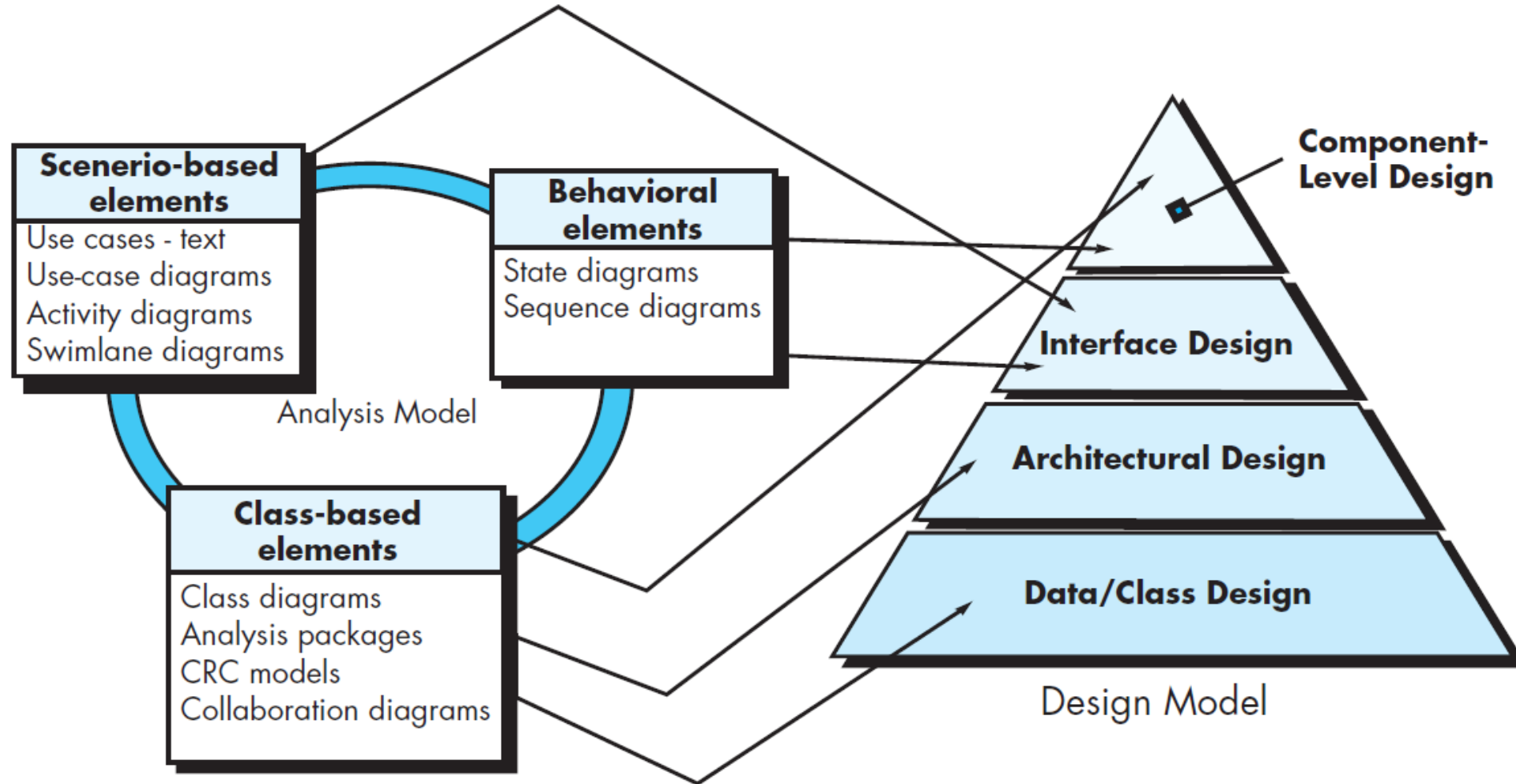
Pressman R S, Bruce R. Maxim, "Software engineering - A Practitioner's Approach", Eighth Edition, Tata McGraw-Hill, 2014
Chapter 13: 13.1, 13.3, 13.6

Topics Covered

Topic	Text Book Reference
Software Architecture	13.1
Architectural Styles	13.3
Architectural Design	13.6

RECAP:

DESIGN WITHIN THE CONTEXT OF SOFTWARE ENGINEERING



Software Architecture

- The software architecture of a program or computing system is the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them
- The architecture is not the operational software. Rather, it is a representation that enables you to
 - analyze the effectiveness of the design in meeting its stated requirements,
 - consider architectural alternatives at a stage when making design changes is still relatively easy, and
 - reduce the risks associated with the construction of the software

Why is architecture important?

Software architecture provides a representation that facilitates communication among all stakeholders

The architecture highlights early design decisions that will have a profound impact on all software engineering work that follows

Architecture “constitutes a relatively small, intellectually graspable model of how the system is structured and how its components work together

Architectural Descriptions

- An architectural description is actually a set of work products that reflect different views of the system
 - The *blueprint metaphor* - Developers regard architecture descriptions as a means of transferring explicit information from architects to designers to software engineers charged with producing the system components
 - The *language metaphor* views architecture as a facilitator of communication across stakeholder groups. This view is preferred by stakeholders with a high customer focus
 - The *decision metaphor* represents architecture as the product of decisions involving trade-offs among properties such as cost, usability, maintainability, and performance
 - The *literature metaphor* is used to document architectural solutions constructed in the past. This view supports the construction of artifacts and the transfer of knowledge between designers and software maintenance staff

Architectural Decisions

INFO



Architecture Decision Description Template

Each major architectural decision can be documented for later review by stakeholders who want to understand the architecture description that has been proposed. The template presented in this sidebar is an adapted and abbreviated version of a template proposed by Tyree and Ackerman [Tyr05].

Design issue:	Describe the architectural design issues that are to be addressed.
Resolution:	State the approach you've chosen to address the design issue.
Category:	Specify the design category that the issue and resolution address (e.g., data design, content structure, component structure, integration, presentation).
Assumptions:	Indicate any assumptions that helped shape the decision.
Constraints:	Specify any environmental constraints that helped shape the decision (e.g., technology standards, available patterns, project-related issues).

Alternatives:

Briefly describe the architectural design alternatives that were considered and why they were rejected.

Argument:

State why you chose the resolution over other alternatives.

Implications:

Indicate the design consequences of making the decision. How will the resolution affect other architectural design issues? Will the resolution constrain the design in any way?

Related decisions:

What other documented decisions are related to this decision?

Related concerns:

What other requirements are related to this decision?

Work products:

Indicate where this decision will be reflected in the architecture description.

Notes:

Reference any team notes or other documentation that was used to make the decision.

Architectural Style

Each style describes a system category that encompasses

1. A set of components (e.g., a database, computational modules) that perform a function required by a system,
2. A set of connectors that enable “communication, coordination and cooperation” among components,
3. Constraints that define how components can be integrated to form the system, and
4. Semantic models that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts

A Brief Taxonomy of Architectural Styles



Data-
centered
Architecture

Data-Flow
Architecture

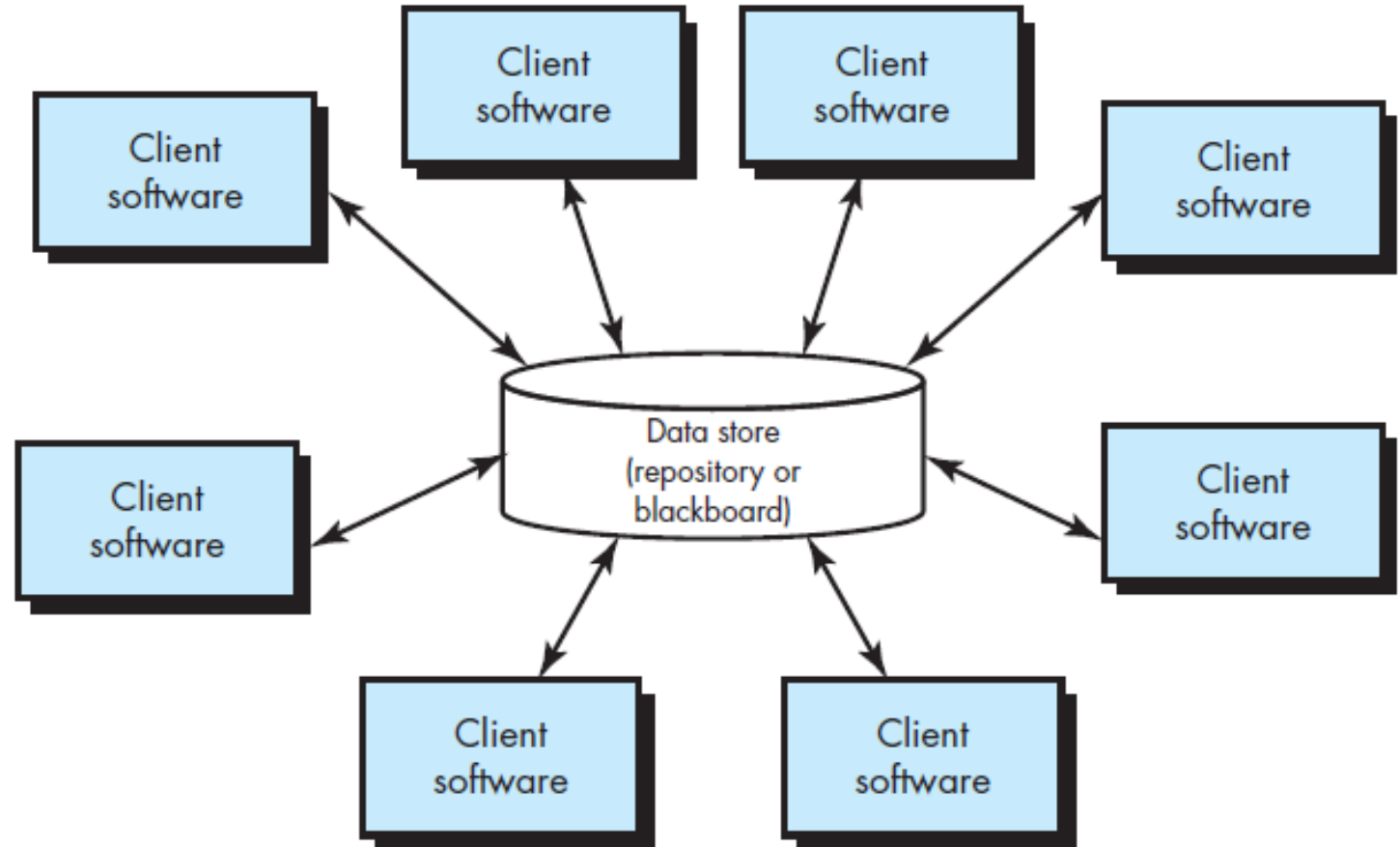
Call-and-
Return
Architecture

Object-
Oriented
Architecture

Layered
Architecture

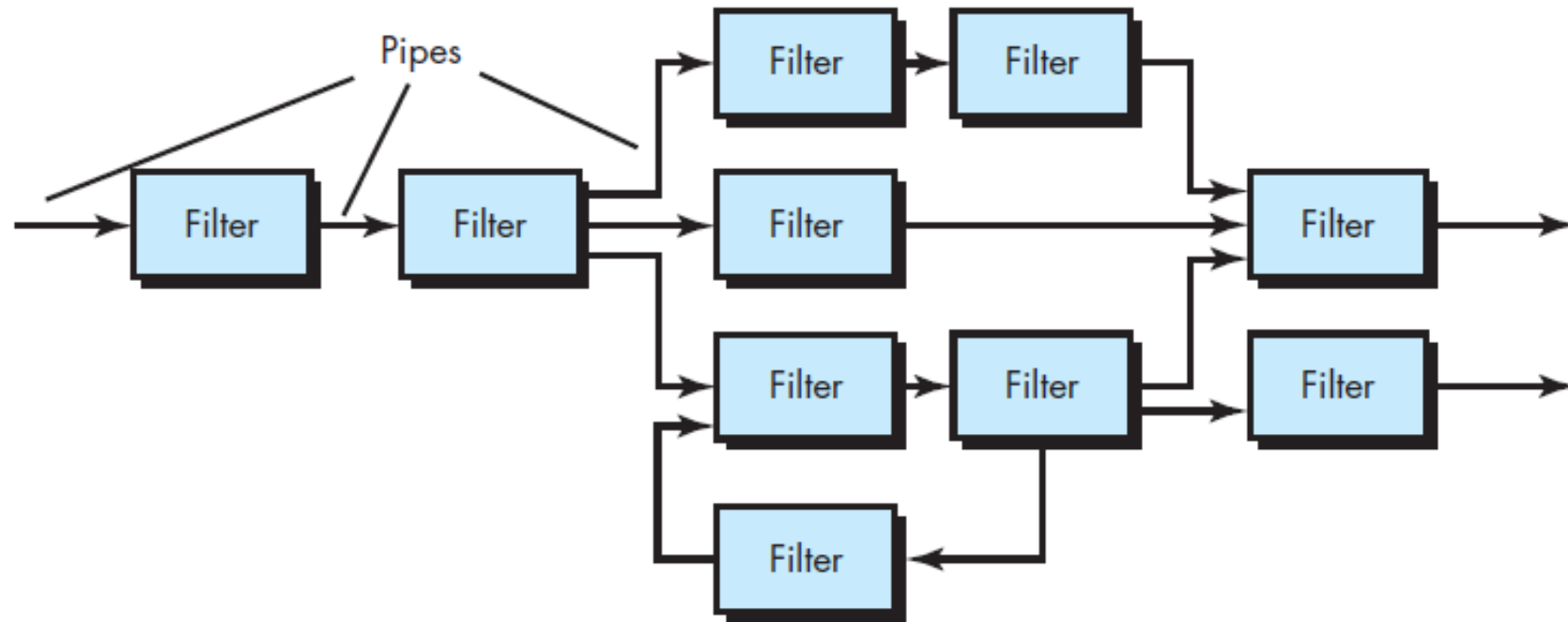
Data-Centered Architecture

- A data store (e.g., a file or database) resides at the center of this architecture and is accessed frequently by other components that update, add, delete, or otherwise modify data within the store



Data-Flow Architectures

- This architecture is applied when input data are to be transformed through a series of computational or manipulative components into output data
- A pipe-and-filter pattern has a set of components, called filters, connected by pipes that transmit data from one component to the next



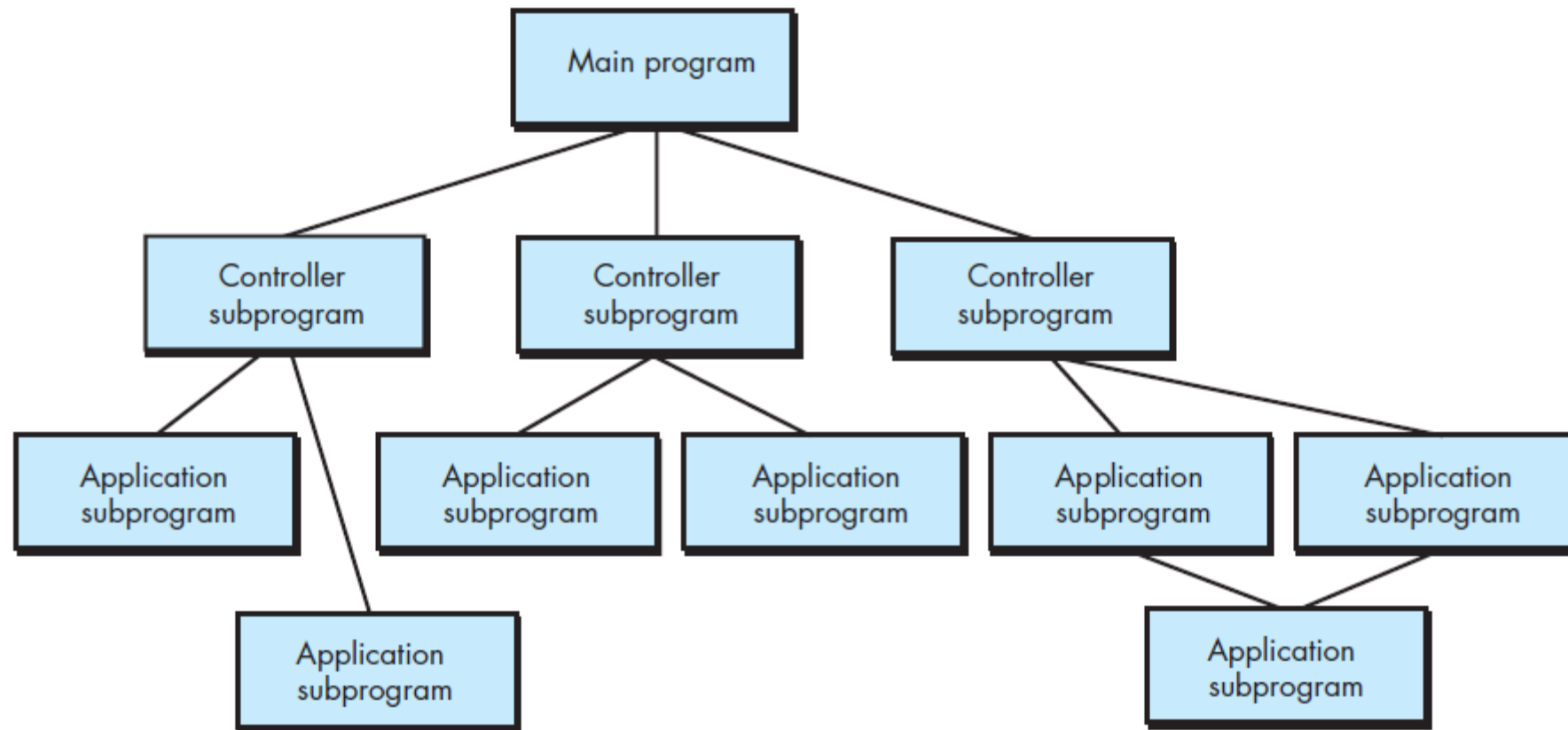
Pipes and filters

Call and Return Architectures

- This architectural style enables you to achieve a program structure that is relatively easy to modify and scale
- *Main program/subprogram architectures*
 - This classic program structure decomposes function into a control hierarchy where a “main” program invokes a number of program components, which in turn may invoke still other components
- *Remote procedure call architectures*
 - The components of a main program/subprogram architecture are distributed across multiple computers on a network

Call and Return Architectures

- *Main program/subprogram architectures*

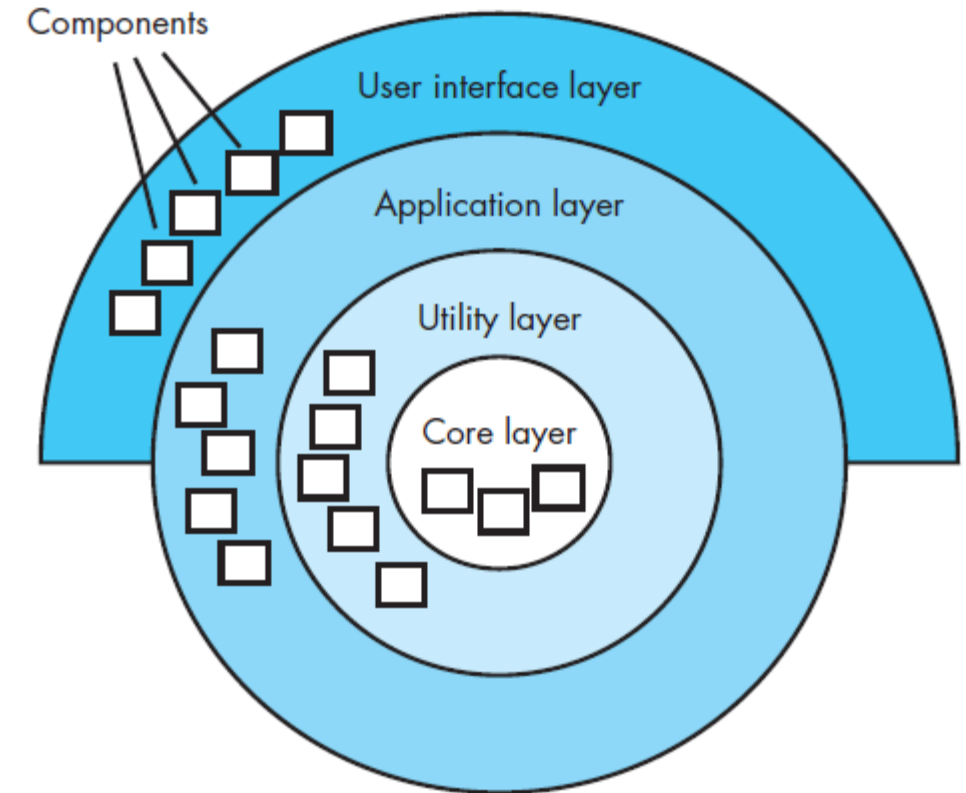


Object-Oriented Architectures

- The components of a system encapsulate data and the operations that must be applied to manipulate the data
- Communication and coordination between components are accomplished via message passing

Layered Architectures

- A number of different layers are defined, each accomplishing operations that progressively become closer to the machine instruction set
- At the outer layer, components service user interface operations. At the inner layer, components perform operating system interfacing
- Intermediate layers provide utility services and application software functions



Question?



Present two or three examples of applications for each of the architectural styles

Architectural Design

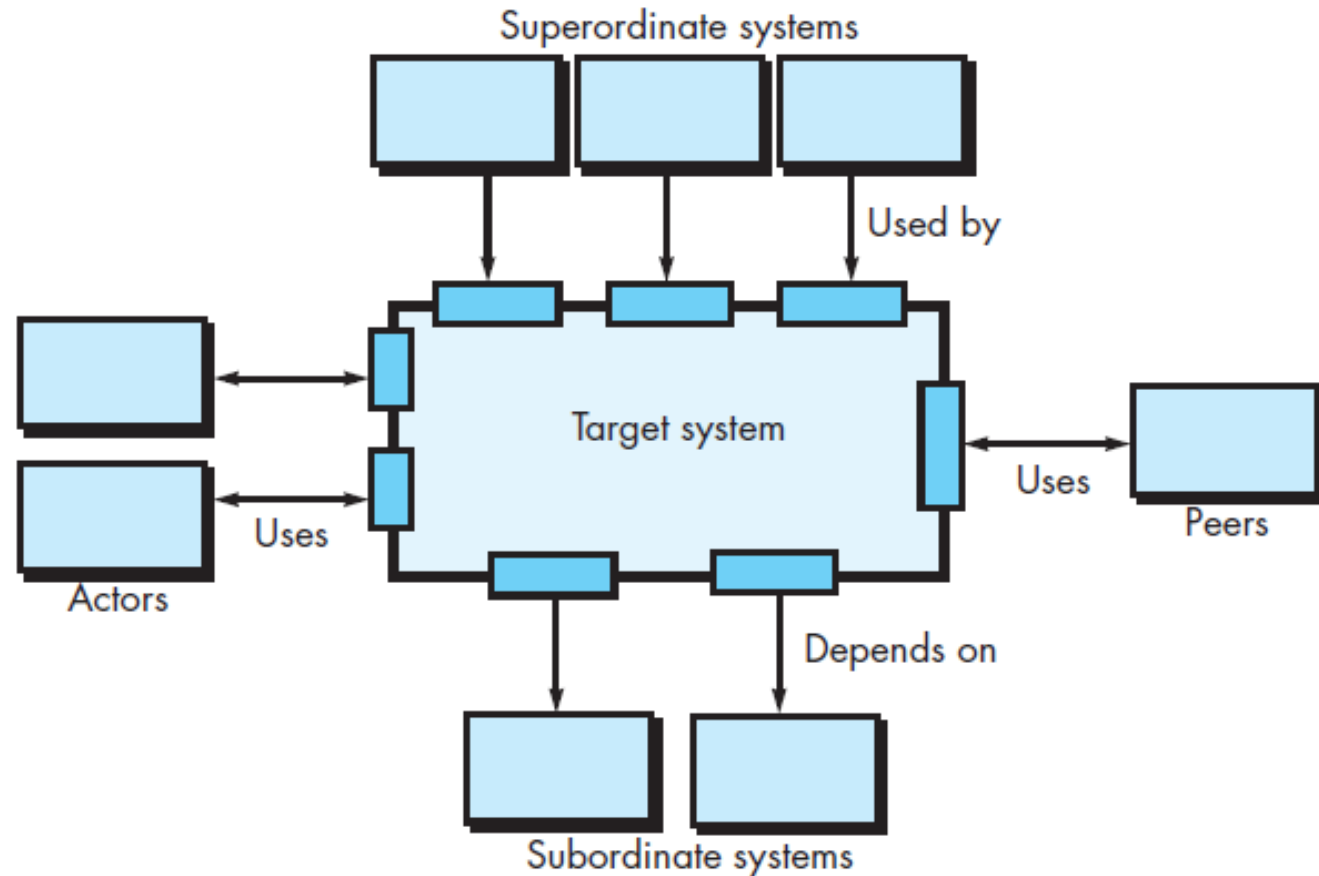
- As architectural design begins, context must be established
- To accomplish this, the external entities (e.g., other systems, devices, people) that interact with the software and the nature of their interaction are described
- This information can generally be acquired from the requirements model
- Once context is modeled and all external software interfaces have been described, you can identify a set of architectural archetypes

Architectural Design

- An *archetype* is an abstraction (similar to a class) that represents one element of system behavior
- The set of archetypes provides a collection of abstractions that must be modeled architecturally if the system is to be constructed, but the archetypes themselves do not provide enough implementation detail
- Therefore, the designer specifies the structure of the system by defining and refining software components that implement each archetype
- This process continues iteratively until a complete architectural structure has been derived

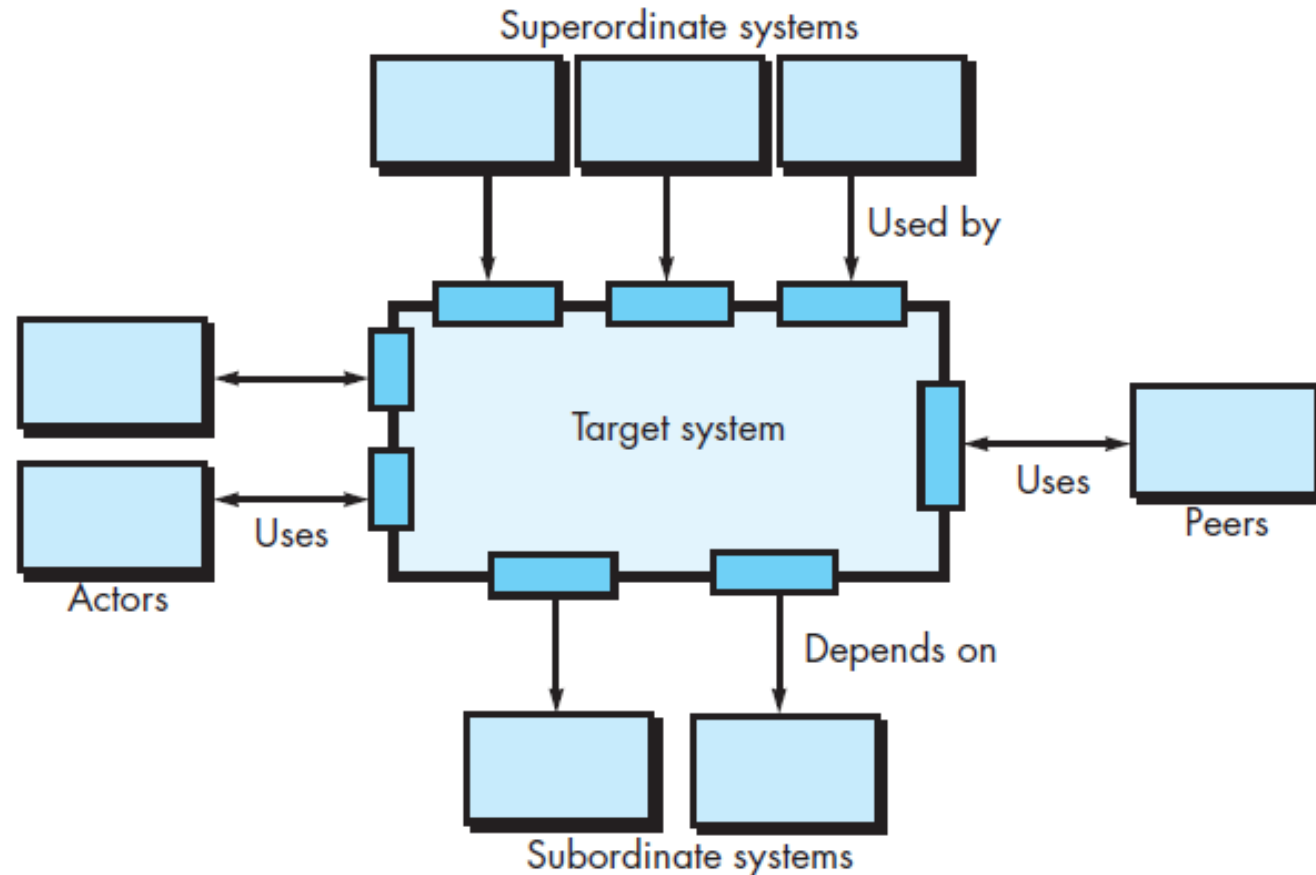
Representing the System in Context

- At the architectural design level, a software architect uses an *architectural context diagram* (ACD) to model the manner in which software interacts with entities external to its boundaries

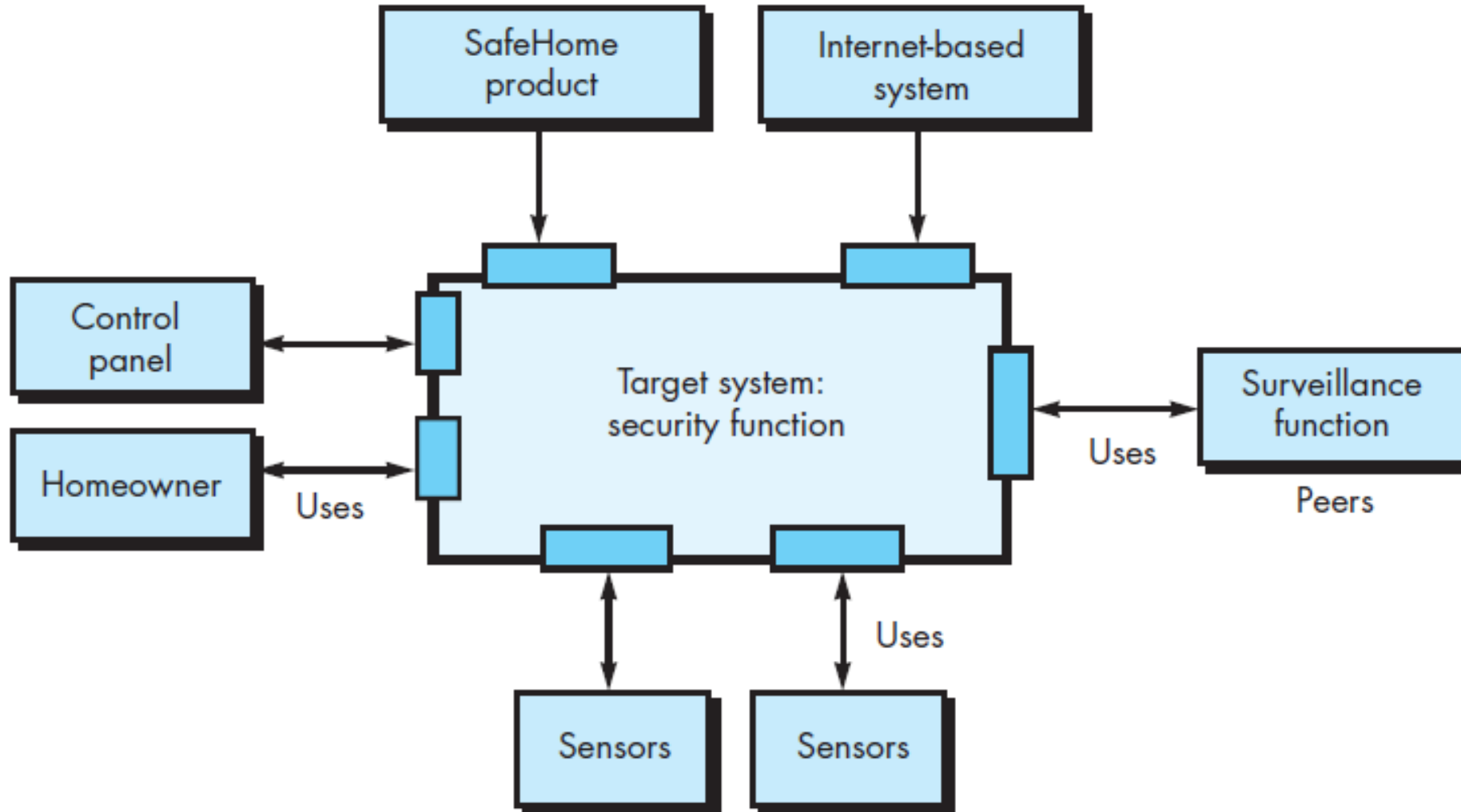


Representing the System in Context

- *Superordinate systems* —those systems that use the target system as part of some higher-level processing scheme
- *Subordinate systems* —those systems that are used by the target system and provide data or processing that are necessary to complete target system functionality
- *Peer-level systems* —those systems that interact on a peer-to-peer basis
- *Actors* —entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing

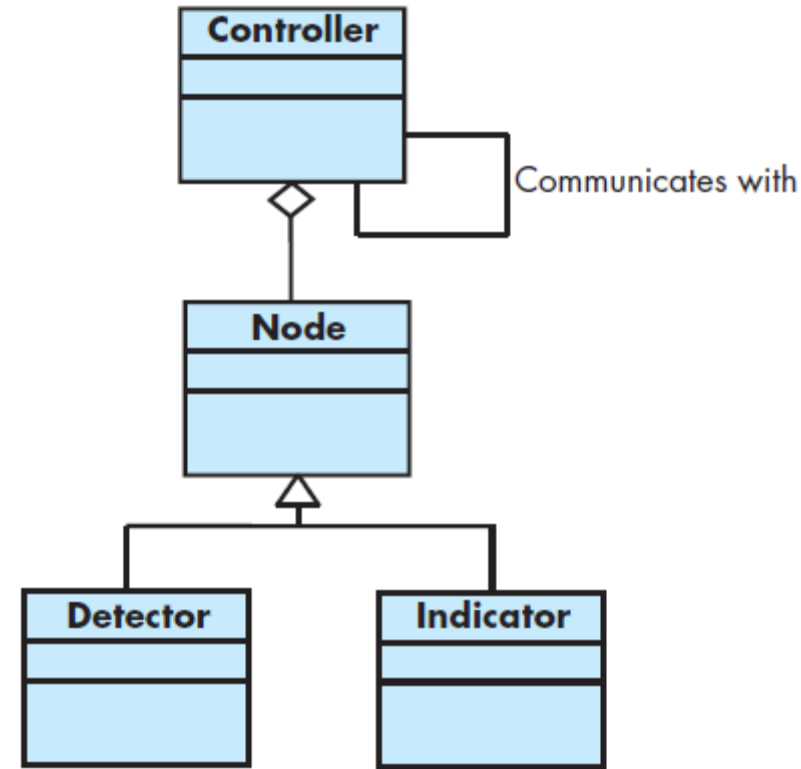


Architectural context diagram for the *SafeHome* security function



Defining Archetypes

- An archetype is a class or pattern that represents a core abstraction that is critical to the design of an architecture for the target system
- In general, a relatively small set of archetypes is required to design even relatively complex systems



UML relationships for *SafeHome* security function archetypes

Refining the architecture into components

- As the software architecture is refined into components, the structure of the system begins to emerge
- The interfaces depicted in the architecture context diagram imply one or more specialized components that process the data that flows across the interface
- In some cases (e.g., a graphical user interface), a complete subsystem architecture with many components must be designed

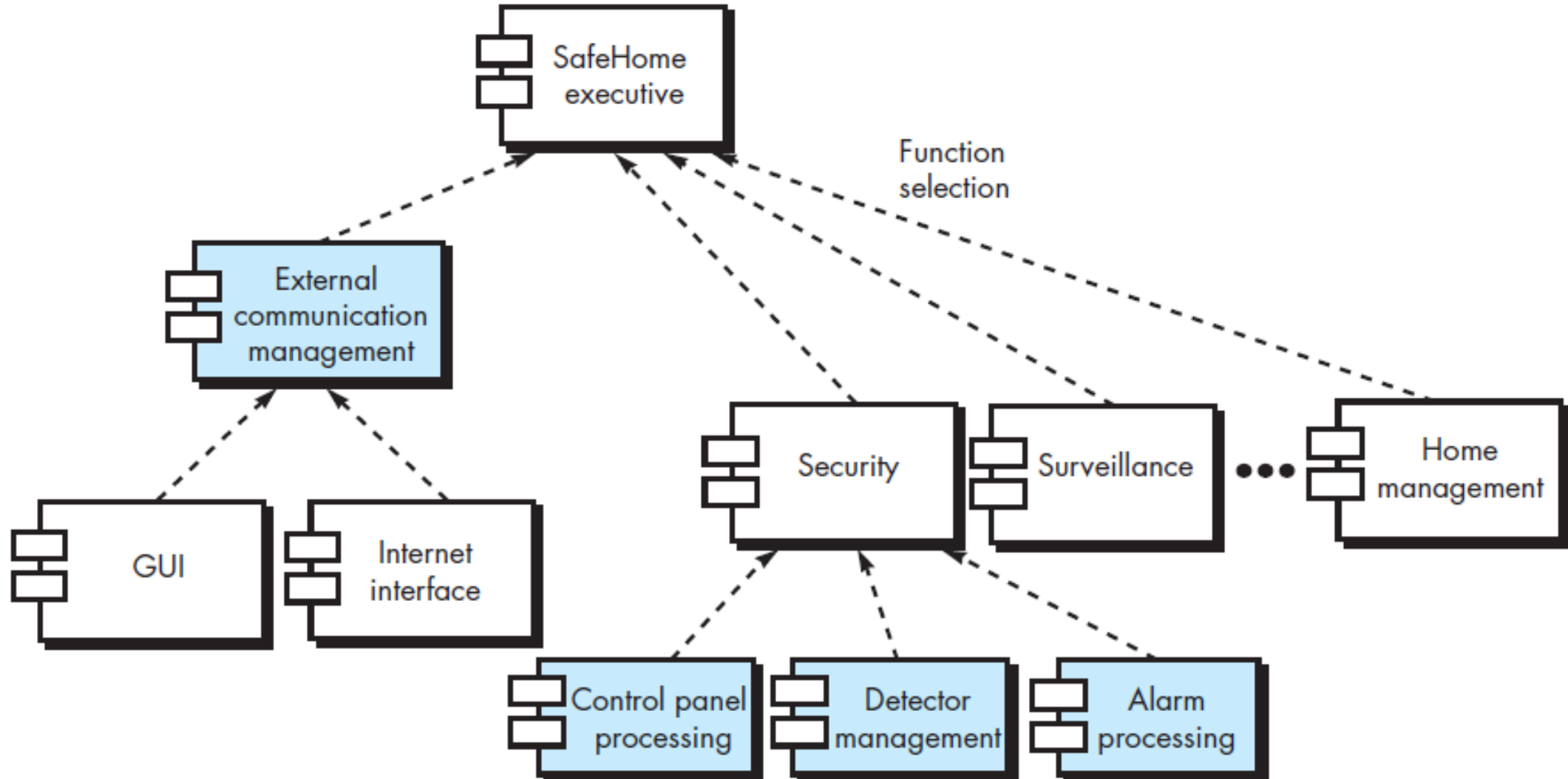
Refining the architecture into components

SafeHome example



- *External communication management* —coordinates communication of the security function with external entities such as other Internet-based systems and external alarm notification
- *Control panel processing* —manages all control panel functionality
- *Detector management* —coordinates access to all detectors attached to the system
- *Alarm processing* —verifies and acts on all alarm conditions

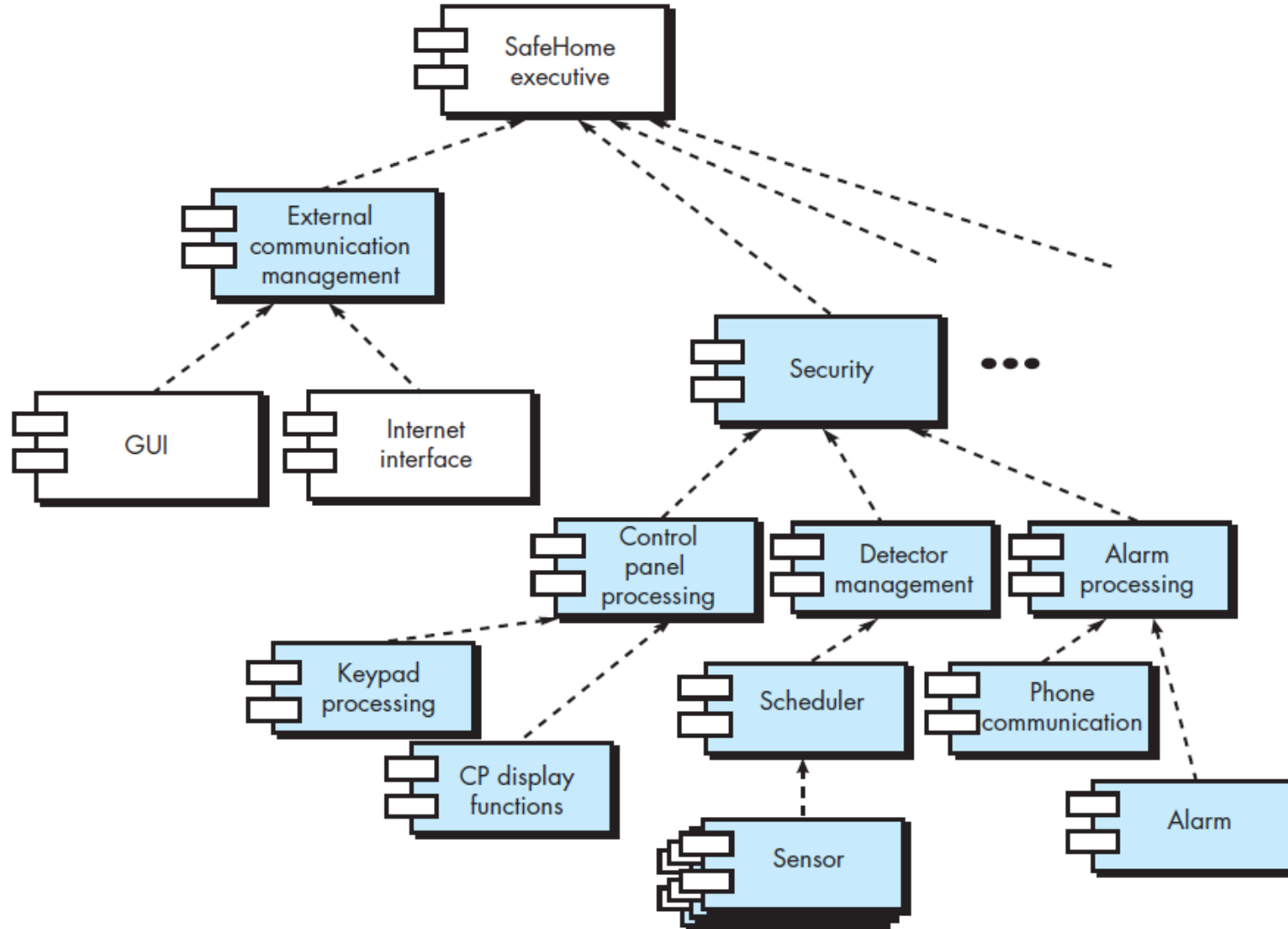
Overall architectural structure for SafeHome with top-level components



Describing Instantiations of the System

- The architectural design that has been modeled to this point is still relatively high level
- For further refinement, an actual instantiation of the architecture is developed
- By this we mean that the architecture is applied to a specific problem with the intent of demonstrating that the structure and components are appropriate

An instantiation of the security function with component elaboration



Summary

- Software architecture provides a holistic view of the system to be built
- It depicts the structure and organization of software components, their properties, and the connections between them
- A number of different architectural styles and patterns are available to the software engineer and may be applied within a given architectural genre

Summary

- In a general sense, architectural design is accomplished using four distinct steps
- First, the system must be represented in context
 - That is, the designer should define the external entities that the software interacts with and the nature of the interaction
- Once context has been specified, the designer should identify a set of top-level abstractions, called archetypes, that represent pivotal elements of the system's behavior or function
- After abstractions have been defined, the design begins to move closer to the implementation domain
- Components are identified and represented within the context of an architecture that supports them
- Finally, specific instantiations of the architecture are developed to “prove” the design in a real-world context